

Documento Técnico — Estruturas de Dados e Algoritmos

Sistema: AV-CAR Auto Center — Gestão de Oficina Mecânica

Disciplina: Estrutura de Dados I

Linguagem: Java 21

Pacote: br.edu.senai.fatesg.avcar.datastructures

Sumário

- [Fila Circular — FilaEsperaOS<T>](#)
- [MergeSort — OrdenacaoOS.mergeSort\(\).](#)
- [QuickSort — OrdenacaoOS.quickSort\(\).](#)
- [Busca com Estrutura de Dados — BuscaOS](#)
- [Funções Recursivas — CalculoOS](#)
- [Integração com a Interface Swing](#)
- [Justificativas Técnicas Consolidadas](#)

1. Fila Circular — FilaEsperaOS<T>

1.1 Onde foi utilizada

A fila circular é utilizada no módulo **Swing** como uma **fila de espera de ordens de serviço**. A oficina pode colocar OSs em espera (ex.: aguardando peça, aguardando liberação do cliente) e retirá-las na ordem FIFO quando estiverem prontas para prosseguir.

A fila é tipada como `FilaEsperaOS<OrdemServicoDT0>`, armazenando objetos completos de ordem de serviço — não apenas identificadores textuais. Isso permite exibir na interface informações como número, placa/veículo e status.

O diálogo `FilaEsperaDialog` (em `swing/views/FilaEsperaDialog.java`) gerencia a fila com interface visual:

- Seleção via ComboBox:** todas as OSs do sistema são carregadas em um `JComboBox<OrdemServicoDT0>` com renderizador customizado exibindo `#NÚMERO | PLACA - MODELO | STATUS`
- Enqueue:** adiciona a OS selecionada ao final da fila
- Dequeue:** remove a OS mais antiga e exibe seus dados na mensagem
- Peek:** consulta a primeira OS sem remover
- Clear:** esvazia a fila

Arquivo: `datastructures/FilaEsperaOS.java`

1.2 Justificativa de escolha

Critério	Decisão
Estrutura	Fila circular com array (<code>Object[]</code>)
Alternativa descartada	<code>LinkedList</code> — embora também ofereça $O(1)$ para enqueue/dequeue, a fila circular com array tem melhor localidade de cache, menor overhead de memória (sem nós individuais) e é didaticamente mais relevante para o curso
Alternativa descartada	<code>PriorityQueue</code> — não atende ao requisito FIFO
Redimensionamento	Dobro da capacidade quando cheia, com rearranjo dos elementos para ordem FIFO contígua
Complexidade	Enqueue/Dequeue/Peek: $O(1)$ amortizado; Listar: $O(n)$

1.3 Trechos relevantes

Estrutura interna e construtor:

```
public class FilaEsperaOS<T> {
    private Object[] elementos;
    private int inicio;
    private int fim;
    private int tamanho;
    private static final int CAPACIDADE_INICIAL = 10;

    public FilaEsperaOS() {
```

```
    this.elementos = new Object[CAPACIDADE_INICIAL];
    this.inicio = 0;
    this.fim = -1;
    this.tamanho = 0;
}
```

Enqueue com atualização circular e redimensionamento automático:

```
public void enqueue(T item) {
    if (isFull()) crescer();
    fim = (fim + 1) % elementos.length;
    elementos[fim] = item;
    tamanho++;
}
```

Dequeue com reaproveitamento de índices via aritmética modular:

```
public T dequeue() {
    if (isEmpty()) throw new IllegalStateException("Fila vazia");
    T item = (T) elementos[inicio];
    elementos[inicio] = null;
    inicio = (inicio + 1) % elementos.length;
    tamanho--;
    return item;
}
```

Redimensionamento com reconstrução FIFO:

```
private void crescer() {
    int novaCapacidade = elementos.length * 2;
    Object[] novoArray = new Object[novaCapacidade];
    for (int i = 0; i < tamanho; i++) {
        novoArray[i] = elementos[(inicio + i) % elementos.length];
    }
    elementos = novoArray;
    inicio = 0;
    fim = tamanho - 1;
}
```

Método listar() — percorre do início ao fim respeitando a circularidade:

```
public List<T> listar() {
    List<T> lista = new ArrayList<>();
    for (int i = 0; i < tamanho; i++) {
        int idx = (inicio + i) % elementos.length;
        lista.add((T) elementos[idx]);
    }
    return lista;
}
```

2. MergeSort — Ordenacao0S.mergeSort()

2.1 Onde foi utilizada

O MergeSort está disponível como método estático no utilitário `Ordenacao0S`. Embora o QuickSort seja usado na interface (por sua ordenação in-place mais rápida), o MergeSort fica disponível como **alternativa estável** para contextos onde a ordem relativa de elementos com chaves iguais precisa ser preservada.

Cenário de uso potencial: relatórios onde itens com mesma data/hora devem manter a ordem de inserção original.

Arquivo: `datastructures/Ordenacao0S.java`

2.2 Justificativa de escolha

Critério	Decisão
Algoritmo	MergeSort clássico (divisão e conquista)
Alternativa descartada	<code>Collections.sort()</code> (TimSort) — proibido pela especificação do trabalho; deve-se implementar manualmente
Complexidade	$O(n \log n)$ garantido (melhor, médio e pior caso)
Estabilidade	Estável — elementos iguais mantêm ordem relativa
Estratégia	Divisão recursiva ao meio, ordenação das metades, intercalação $O(n)$ com arrays auxiliares

2.3 Trechos relevantes

Método público e recursão principal:

```
public static <T> void mergeSort(List<T> lista, Comparator<T> comparator) {
    if (lista == null || lista.size() <= 1) return;
    mergeSortRecursivo(lista, 0, lista.size() - 1, comparator);
}

private static <T> void mergeSortRecursivo(List<T> lista, int left, int right, Comparator<T> c) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    mergeSortRecursivo(lista, left, mid, c);
    mergeSortRecursivo(lista, mid + 1, right, c);
    merge(lista, left, mid, right, c);
}
```

Método de intercalação (merge) com arrays auxiliares:

```
private static <T> void merge(List<T> lista, int left, int mid, int right, Comparator<T> c) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    Object[] leftArr = new Object[n1];
    Object[] rightArr = new Object[n2];

    for (int i = 0; i < n1; i++) leftArr[i] = lista.get(left + i);
    for (int j = 0; j < n2; j++) rightArr[j] = lista.get(mid + 1 + j);

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        T a = (T) leftArr[i];
        T b = (T) rightArr[j];
        if (c.compare(a, b) <= 0) {
            lista.set(k++, a);
            i++;
        } else {
            lista.set(k++, b);
            j++;
        }
    }
    while (i < n1) lista.set(k++, (T) leftArr[i++]);
    while (j < n2) lista.set(k++, (T) rightArr[j++]);
}
```

3. QuickSort — Ordenacao0S.quickSort()

3.1 Onde foi utilizada

O QuickSort é utilizado na **interface gráfica Swing** para ordenar a tabela de Ordens de Serviço quando o usuário clica nos cabeçalhos das colunas (`OrdemServicoPanel.java`).

O `MouseListener` no cabeçalho da `JTable` detecta o clique, identifica a coluna e ordena os dados usando `OrdenacaoOS.quickSort()` :

```
tabela.getTableHeader().addMouseListener(new MouseAdapter() {
    @Override
    public void mouseClicked(MouseEvent e) {
        int col = tabela.columnAtPoint(e.getPoint());
        if (col == sortColumn) sortAsc = !sortAsc;
        else { sortColumn = col; sortAsc = true; }
        ordenarTabela(col);
    }
});
```

3.2 Justificativa de escolha

Critério	Decisão
Algoritmo	QuickSort com Lomuto partition e mediana como pivô
Alternativa descartada	MergeSort — embora estável, requer $O(n)$ memória extra; para ordenação in-place na UI, QuickSort é mais eficiente
Por que QuickSort na UI	O usuário espera resposta rápida ao clicar no cabeçalho; QuickSort é $O(n \log n)$ médio e ordena in-place sem alocar arrays auxiliares grandes
Pivô	Mediana $(low + high) / 2$ — reduz chance de pior caso $O(n^2)$ em dados parcialmente ordenados
Instabilidade	Instável, mas irrelevante para a UI (dados exibidos em tabela)

3.3 Trechos relevantes

Método público e recursão principal:

```
public static <T> void quickSort(List<T> lista, Comparator<T> comparator) {
    if (lista == null || lista.size() <= 1) return;
    quickSortRecursivo(lista, 0, lista.size() - 1, comparator);
}

private static <T> void quickSortRecursivo(List<T> lista, int low, int high, Comparator<T> c) {
    if (low >= high) return;
    int pi = partition(lista, low, high, c);
    quickSortRecursivo(lista, low, pi - 1, c);
    quickSortRecursivo(lista, pi + 1, high, c);
}
```

Lomuto partition com pivô mediano:

```
private static <T> int partition(List<T> lista, int low, int high, Comparator<T> c) {
    int mid = low + (high - low) / 2;
    T pivot = lista.get(mid);

    T temp = lista.get(mid);
    lista.set(mid, lista.get(high));
    lista.set(high, temp);

    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (c.compare(lista.get(j), pivot) <= 0) {
            i++;
            T troca = lista.get(i);
            lista.set(i, lista.get(j));
            lista.set(j, troca);
        }
    }
    T troca = lista.get(i + 1);
    lista.set(i + 1, lista.get(high));
```

```
        lista.set(high, troca);
        return i + 1;
    }
}
```

Uso na UI — mapeamento de colunas para comparadores:

```
private void ordenarTabela(int col) {
    Comparator<OrdemServicoDTO> comp = switch (col) {
        case 0 -> Comparator.comparing(OrdemServicoDTO::getId);
        case 1 -> Comparator.comparing(OrdemServicoDTO::getNumero);
        case 5 -> Comparator.comparing(OrdemServicoDTO::getStatus);
        case 6 -> Comparator.comparing(OrdemServicoDTO::getDataAbertura);
        case 8 -> Comparator.comparing(OrdemServicoDTO::getValorTotal);
        // ... demais colunas
    };
    if (!sortAsc) comp = comp.reversed();
    OrdenacaoOS.quickSort(dadosCarregados, comp);
    // re-renderiza a tabela
}
```

4. Busca com Estrutura de Dados — BuscaOS

4.1 Onde foi utilizada

A classe `BuscaOS` implementa dois algoritmos de busca que são utilizados na **interface gráfica Swing** para localizar rapidamente Ordens de Serviço na tabela principal (`OrdemServicoPanel.java`). A toolbar do painel de OS contém um campo de texto e um botão "Buscar":

- Ao digitar um termo (placa do veículo, número da OS, status, etc.), a **busca linear** percorre a lista carregada e seleciona a primeira ocorrência que contenha o termo.
- Quando o termo é numérico e corresponde exatamente a um identificador, a **busca binária** pode ser utilizada sobre a lista ordenada para localização em $O(\log n)$.

Arquivo: `datastructures/BuscaOS.java`

4.2 Justificativa de escolha

Critério	Decisão
Busca Linear	Adequada para busca textual parcial (contém) — o usuário pode digitar parte da placa, nome ou número — não exige pré-ordenação
Busca Binária	Ideal para busca exata em listas ordenadas (ex.: localizar OS por ID após ordenação), com complexidade $O(\log n)$
Alternativa descartada	Map/HashMap — embora ofereça $O(1)$, não demonstra implementação manual exigida; <code>Stream.filter</code> — proibido
Estratégia para busca textual	Normalização para lowercase + <code>contains()</code> — permite buscas parciais case-insensitive
Complexidade	Busca Linear: $O(n)$; Busca Binária: $O(\log n)$

4.3 Trechos relevantes

Busca Linear com extrator de campo genérico:

```
public static <T> int buscaLinear(List<T> lista, Function<T, String> extrator, String alvo) {
    if (lista == null || alvo == null) return -1;
    String target = alvo.toLowerCase();
    for (int i = 0; i < lista.size(); i++) {
        String valor = extrator.apply(lista.get(i));
        if (valor != null && valor.toLowerCase().contains(target)) {
            return i;
        }
    }
    return -1;
}
```

Busca Binária com Comparador genérico:

```
public static <T> int buscaBinaria(List<T> lista, T alvo, Comparator<T> comparator) {
    if (lista == null || lista.isEmpty()) return -1;
    int esq = 0, dir = lista.size() - 1;
    while (esq <= dir) {
        int meio = esq + (dir - esq) / 2;
        int cmp = comparator.compare(lista.get(meio), alvo);
        if (cmp == 0) return meio;
        if (cmp < 0) esq = meio + 1;
        else dir = meio - 1;
    }
    return -1;
}
```

Uso na UI — campo de busca na toolbar:

```
JTextField tfBusca = new JTextField(12);
JButton btnBuscar = new JButton("Buscar");
toolbar.add(new JLabel("Buscar:"));
toolbar.add(tfBusca);
toolbar.add(btnBuscar);

btnBuscar.addActionListener(e -> buscarOS(tfBusca.getText().trim()));
tfBusca.addActionListener(e -> buscarOS(tfBusca.getText().trim()));
```

Método buscarOS — integra a busca linear com a tabela:

```
private void buscarOS(String termo) {
    if (termo.isEmpty()) { carregarDados(); return; }
    int idx = BuscaOS.buscaLinear(dadosCarregados,
        os -> os.getVeiculo() + " " + os.getNumeroOs() + " " + os.getStatus(),
        termo);
    if (idx >= 0) {
        tabela.setRowSelectionInterval(idx, idx);
        tabela.scrollRectToVisible(tabela.getCellRect(idx, 0, true));
    } else {
        JOptionPane.showMessageDialog(this,
            "Nenhuma OS encontrada para \"" + termo + "\".");
    }
}
```

5. Funções Recursivas — CalculoOS

5.1 Onde foi utilizada

As funções recursivas do CalculoOS são utilizadas no diálogo ItensOSDialog (gerenciamento de itens de serviço, peças e serviços externos de uma OS) para exibir os totais no rodapé da janela.

O método atualizarTotal() extrai os valores das tabelas, alimenta listas de Double e invoca CalculoOS.somarValores() (recursivo) para cada categoria, além de CalculoOS.calcularValorTotal() para o total geral com desconto.

A classe CalculoOS contém funções puramente recursivas que demonstram o paradigma de recursão aplicado a problemas reais da oficina:

Função	Propósito
somarValores(List<Double>, int)	Soma recursiva de valores de itens (serviços, peças, serviços externos)
calcularValorTotal(...)	Calcula o valor total de uma OS usando somarValores() recursivo para cada categoria e aplica desconto
fatorial(int)	Calcula permutações de agendamento (ex.: número de formas de ordenar N serviços no dia)

Arquivo: datastructures/CalculoOS.java

4.2 Justificativa de escolha

Critério	Decisão
Paradigma	Recursão pura (sem iteração)
Alternativa descartada	Laço for iterativo — mais eficiente, mas não demonstra o conceito de recursão exigido
Caso base	índice >= lista.size() para somarValores; n <= 1 para fatorial
Passo recursivo	lista.get(índice) + somarValores(lista, índice + 1) para soma; n * fatorial(n - 1) para fatorial
Aplicação real	somarValores é usado por calcularValorTotal que integra todas as três categorias de custo de uma OS

5.3 Trechos relevantes

Integração no ItensOSDialog — extração de valores e chamada recursiva:

```
private void atualizarTotal() {
    List<Double> valsServicos = new ArrayList<>();
    for (int i = 0; i < modelServicos.getRowCount(); i++)
        valsServicos.add(parseValor(modelServicos.getValueAt(i, 4)));

    List<Double> valsPecas = new ArrayList<>();
    for (int i = 0; i < modelPecas.getRowCount(); i++)
        valsPecas.add(parseValor(modelPecas.getValueAt(i, 4)));

    List<Double> valsExternos = new ArrayList<>();
    for (int i = 0; i < modelExternos.getRowCount(); i++)
        valsExternos.add(parseValor(modelExternos.getValueAt(i, 3)));

    double total = CalculoOS.calcularValorTotal(valsServicos, valsPecas, valsExternos, 0);
    lblTotal.setText(String.format(
        "Serviços: R$ %.2f | Peças: R$ %.2f | Externos: R$ %.2f | TOTAL: R$ %.2f",
        CalculoOS.somarValores(valsServicos, 0),
        CalculoOS.somarValores(valsPecas, 0),
        CalculoOS.somarValores(valsExternos, 0), total));
}
```

Somador recursivo — percorre a lista do índice até o final:

```
public static double somarValores(List<Double> valores, int i) {
    if (i >= valores.size()) return 0;
    return valores.get(i) + somarValores(valores, i + 1);
}
```

Cálculo do valor total usando a recursão:

```
public static double calcularValorTotal(
    List<Double> servicos,
    List<Double> pecas,
    List<Double> externos,
    double valorDesconto) {
    double totalServicos = somarValores(servicos, 0);
    double totalPecas = somarValores(pecas, 0);
    double totalExternos = somarValores(externos, 0);
    double total = totalServicos + totalPecas + totalExternos;
    return total - Math.min(total, valorDesconto);
}
```

Fatorial recursivo clássico:

```
public static long fatorial(int n) {
    if (n <= 1) return 1;
    return n * fatorial(n - 1);
}
```

```
return n * fatorial(n - 1);  
}
```

6. Integração com a Interface Swing

6.1 Fila de Espera — FilaEsperaDialog

O diálogo `FilaEsperaDialog` utiliza uma instância **estática** de `FilaEsperaOS<OrdemServicoDTO>` que persiste durante toda a sessão do Swing. Diferentemente da versão anterior (que armazenava apenas `String` com o número da OS), a fila agora armazena objetos completos de `OrdemServicoDTO`, permitindo exibir na interface informações como número, veículo e status.

A seleção da OS a ser enfileirada é feita por um `JComboBox<OrdemServicoDTO>` com renderizador customizado, populado a partir do `OrdemServicoController.listar()`.

O botão **"Fila de Espera"** na toolbar do painel de Ordens de Serviço (`OrdemServicoPanel`) agora passa o controller para o diálogo:

```
// OrdemServicoPanel.java  
JButton btnFila = new JButton("Fila de Espera");  
toolbar.add(btnFila);  
btnFila.addActionListener(e ->  
    FilaEsperaDialog.showDialog(SwingUtilities.getWindowAncestor(this),  
        ordemServicoController));
```

Renderizador customizado do ComboBox:

```
comboOS.setRenderer(new DefaultListCellRenderer() {  
    @Override  
    public Component getListCellRendererComponent(...) {  
        if (value instanceof OrdemServicoDTO os) {  
            setText(String.format("#%d | %s | %s",  
                os.getNumeroOs(), os.getVeiculo(), os.getStatus()));  
        }  
        return c;  
    }  
});
```

6.2 Ordenação por Cabeçalho de Tabela

O clique no cabeçalho da `JTable` dispara `OrdenacaoOS.quickSort()` sobre a lista de DTOs carregados. O mapeamento de colunas para `Comparator` utiliza os getters do `OrdemServicoDTO`:

```
private void ordenarTabela(int col) {  
    Comparator<OrdemServicoDTO> comp = switch (col) {  
        case 0 -> Comparator.comparing(OrdemServicoDTO::getNumeroOs,  
            Comparator.nullsLast(Comparator.naturalOrder()));  
        case 1 -> Comparator.comparing(OrdemServicoDTO::getVeiculo,  
            Comparator.nullsLast(String::compareTo));  
        case 2 -> Comparator.comparing(OrdemServicoDTO::getStatus,  
            Comparator.nullsLast(String::compareTo));  
        case 3 -> Comparator.comparing(OrdemServicoDTO::getDataAbertura,  
            Comparator.nullsLast(Comparator.naturalOrder()));  
        case 4 -> Comparator.comparing(OrdemServicoDTO::getEntradaVeiculo,  
            Comparator.nullsLast(Comparator.naturalOrder()));  
        case 5 -> Comparator.comparing(OrdemServicoDTO::getValorTotal,  
            Comparator.nullsLast(Comparator.naturalOrder()));  
        case 6 -> Comparator.comparing(OrdemServicoDTO::getValorDesconto,  
            Comparator.nullsLast(Comparator.naturalOrder()));  
    };  
    if (!sortAsc) comp = comp.reversed();  
    OrdenacaoOS.quickSort(dadosCarregados, comp);  
    modelo.setRowCount(0);  
}
```



```
        for (var os : dadosCarregados) modelo.addRow(linha(os));
    }
}
```

6.3 Busca na Barra de Ferramentas

O campo de busca utiliza `BuscaOS.buscaLinear()` para localizar a primeira OS que contenha o termo digitado (por placa/veículo, número ou status). Quando encontrada, a linha é selecionada e a tabela rola automaticamente até ela.

6.4 Cálculo Recursivo no Gerenciamento de Itens — `ItensOSDialog`

O diálogo `ItensOSDialog`, responsável por gerenciar os itens de serviço, peças e serviços externos de uma ordem de serviço, exibe no **rodapé da janela** os totais calculados exclusivamente por funções recursivas do `CalculoOS`.

O método `atualizarTotal()` é chamado automaticamente após cada operação (adicionar ou remover item) nos três métodos de carregamento (`carregarServicos()`, `carregarPecas()`, `carregarExternos()`):

```
private void atualizarTotal() {
    // Extrai valores das tabelas e alimenta listas
    List<Double> valsServicos = /* ... */;
    List<Double> valsPecas = /* ... */;
    List<Double> valsExternos = /* ... */;

    // Invoca funções recursivas do CalculoOS
    double total = CalculoOS.calcularValorTotal(
        valsServicos, valsPecas, valsExternos, 0);

    lblTotal.setText(String.format(
        "Serviços: R$ %.2f | Peças: R$ %.2f | " +
        "Externos: R$ %.2f | TOTAL: R$ %.2f",
        CalculoOS.somarValores(valsServicos, 0),
        CalculoOS.somarValores(valsPecas, 0),
        CalculoOS.somarValores(valsExternos, 0),
        total));
}
```

7. Justificativas Técnicas Consolidadas

Mapa de Decisões

Estrutura/Algoritmo	Problema resolvido	Por que esta escolha	O que foi descartado
Fila Circular	Gerenciar OSs em espera na oficina	FIFO natural para fila de espera; $O(1)$ enqueue/dequeue; array com redimensionamento dinâmico	LinkedList (overhead de memória); PriorityQueue (não é FIFO)
MergeSort	Ordenação estável de dados	$O(n \log n)$ garantido; estável; bom para relatórios	<code>Collections.sort()</code> (proibido); QuickSort (instável)
QuickSort	Ordenação in-place na UI	$O(n \log n)$ médio; ordena in-place; resposta rápida ao clique do usuário	MergeSort (memória extra $O(n)$); BubbleSort ($O(n^2)$ lento)
Busca Linear	Localizar OS por placa, número ou status	Busca textual parcial case-insensitive; não exige pré-ordenação; $O(n)$ direto	HashMap (não demonstra implementação manual); <code>Stream.filter</code> (proibido)
Busca Binária	Localizar OS exata em lista ordenada	$O(\log n)$ eficiente; didático para ensino de divisão e conquista	<code>Collections.binarySearch</code> (proibido)
Recursão (soma)	Cálculo de totais de OS no <code>ItensOSDialog</code>	Didático para demonstrar recursão; caso base + passo recursivo claros; integrado ao rodapé da interface	Laço <code>for</code> (não demonstra recursão)
Recursão (fatorial)	Permutações de agendamento	Função clássica para ensino de recursão	Iteração (não atende ao requisito)

Complexidades

Operação	FilaEsperaOS	MergeSort	QuickSort	BuscaOS	CalculoOS
Inserção	O(1)*	—	—	—	—
Remoção	O(1)	—	—	—	—
Consulta	O(1) peek	—	—	—	—
Ordenação	—	O(n log n)	O(n log n) médio	—	—
Busca Linear	—	—	—	O(n)	—
Busca Binária	—	—	—	O(log n)	—
Soma	—	—	—	—	O(n)
Fatorial	—	—	—	—	O(n)

*O(1) amortizado — O(n) quando o array precisa ser redimensionado

Conformidade com as Restrições

- Nenhum algoritmo usa `Collections.sort` , `Arrays.sort` , `Collections.binarySearch` , `Stream.sorted` ou `List.sort`
- Todas as implementações usam apenas estruturas básicas da linguagem (arrays, loops, recursão)
- A fila circular é implementada com `Object[]` e aritmética modular, sem uso de `LinkedList` ou filas prontas da `java.util`
- As funções recursivas são implementadas sem iteração (sem laços `for` / `while`)
- Os algoritmos de busca utilizam apenas arrays e comparações diretas, sem `Map` , `HashSet` ou streams

Documento gerado em Junho de 2026 para a disciplina de Estrutura de Dados I.
Código-fonte disponível em `src/main/java/br/edu/senai/fatesg/avcar/datastructures/`